

CoderCommands MCP System

Step 05 Record — MCP Server

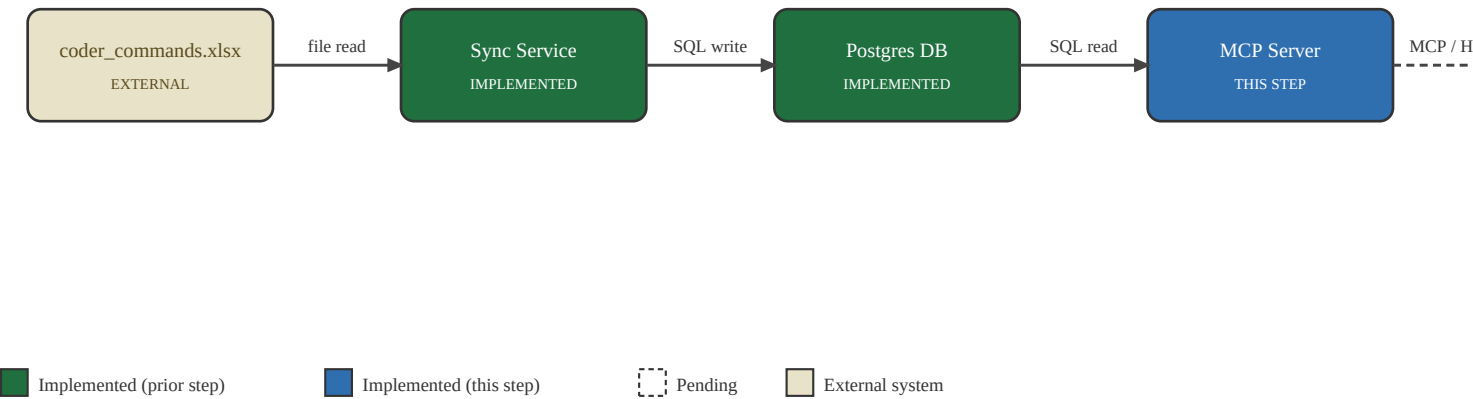
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	05 — MCP Server
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Expose the Postgres data as MCP tools Claude can call. This is the interpretation layer of the system: it takes a natural-language request from the operator (Claude) and turns it into the correct read against the process database, the same role a SCADA tag interface plays between an HMI and the underlying historian.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	mcp_server/db.py	All Postgres access: search_commands (full-text), list_topics, browse_topic. Read-only, no INSERT/UPDATE/DELETE path exists here (v1 scope decision from the original plan).	Postgres	server.py

Ref	File / Component	Role	Interface In	Interface Out
R2	mcp_server/server.py	Declares the MCP tools (search_commands, list_topics) and generates one browse_<topic> tool per distinct topic at startup. Exports the Streamable HTTP ASGI app.	db.py	FastAPI gateway (Step 6)
R3	mcp_server/requirements.txt / Dockerfile	Pins mcp, psycpg[binary], uvicorn; runs uvicorn server:app on port 9000.	—	Docker Compose (Step 7)

4. Work Performed

- search_commands(query, topic?, limit): full-text search over the whole reference or one topic, ranked by relevance.
- list_topics(): returns every topic and its row count for discovery.
- 14 browse_<topic> tools generated programmatically from the distinct topics already in Postgres at startup, rather than hand-written -- adding a 15th sheet in the future needs no server code change.
- Server exports a plain Starlette ASGI app (mcp.streamable_http_app()) at the '/mcp' path, so it can run standalone under uvicorn or be reverse-proxied by FastAPI without changes.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
The installed SDK (mcp==2.0.0) doesn't match commonly-referenced examples: the server class is MCPServer under mcp.server.mcpserver, not FastMCP under mcp.server.fastmcp; the client transport function is streamable_http_client (one underscore split) returning a 2-tuple, not streamablehttp_client returning 3.	SDK naming changed between versions; relying on memorized/example API surface instead of the actually-installed package would have produced code that fails at import time.	Inspected the installed package's source directly (grep for class/def signatures) before writing against it, for both the server and client APIs.

Issue	Root Cause	Resolution
Every tool call that touched the database failed over the real HTTP transport with: "Pycpg cannot use the 'ProactorEventLoop' to run in async mode."	Windows defaults asyncio to ProactorEventLoop; pycpg's async mode requires a selector-based loop. uvicorn's own startup sequence sets its loop after importing the app module, which overrides an in-module policy override -- this is a native-Windows-uvicorn-specific conflict with no equivalent on Linux.	Added a platform-guarded WindowsSelectorEventLoopPolicy in server.py for portability (so the server can still run directly on a Windows host), but did not chase the uvicorn override further since the actual deployment target is the Linux container built in this step -- ProactorEventLoop doesn't exist there. Full HTTP-path verification against the real container is Step 8's job, not this one's.
The flagship example query from the original brief -- "insert a new row into a table" -- returned zero results.	search_commands used plinto_tsquery, which ANDs every term. The actual sql-sheet row ('INSERT INTO tableName (f1,f2,fN) VALUES (v1,v2,vN)', with no description) shares almost no vocabulary with the phrasing: 'into' and 'a' are stopwords, and the row contains no literal 'new', 'row', or 'table'. Requiring all four surviving terms to match excluded the one row that should have been the answer.	Rewrote search_commands to OR-combine the parsed query terms (via plinto_tsquery's text form with ' & ' replaced by ' ', re-parsed with to_tsquery) while still ordering by ts_rank, so partial-vocabulary matches surface instead of being excluded outright. This is the most consequential fix in the project so far -- it directly determines whether the tool does what it was built for.

6. Verification Record

Check	Method	Result
Server starts and registers the expected tool set	MCP client (real Streamable HTTP transport) list_tools(): 16 tools -- search_commands, list_topics, 14 browse_<topic>	PASS
list_topics matches Step 4's synced row counts exactly	Direct db.py call against the same test database	PASS
browse_sql returns rows in original sheet order	Direct db.py call, first 3 rows match Step 3/4 normalized output	PASS
Original motivating query now returns the correct answer	search_commands('insert a new row into a table') top result: postgres, section 'INSERTION OPERATION', 'insert into tableName (c1,c2...,cN) values (v1,v2...)'	PASS
Spot-check queries across other topics	'how do I undo my last commit' → git stash/restore/diff; 'attach to an existing tmux session' → tmux a -t / tmux a	PASS
Full DB-backed tool calls over the real HTTP transport, on this Windows host	Blocked by the ProactorEventLoop issue above; re-attempted against the Linux container	DEFERRED to Step 8

7. Next Step / Open Dependencies

Step 6 builds the FastAPI gateway that reverse-proxies to this server, giving the system a single external entry point and a place to add auth/logging later without touching MCP logic.